



Department of Computer Science
Lahore Garrison University, Pakistan



Computer Architecture

Project Report

Submitted by:

Sr. No	Name	Roll No.	Section
01	Saifullah	Fa-2023-BSCS-158	D
02	Asad Ali	Fa-2023-BSCS-149	D
03	M. Hassan Kashif	Fa-2023-BSCS-158	D
04	M. Saud Ali	Fa-2023-BSCS-132	D
05	Salman Irfan	Fa-2023-BSCS-603	D

Submitted to:

Name	Sir Ahsan Raza
------	----------------

Lahore Garrison University

Main Campus, Sector-C Phase-VI, DHA Lahore



Cipher Encryption / Decryption System

Using MIPS Architecture

1. Project Overview

This project implements a **Simple Encryption and Decryption System** using **MIPS Assembly Language**.

The system uses a **Caesar Cipher (Shift Cipher)** to encrypt and decrypt user messages. Only **alphabetic characters** are encrypted, while **spaces, numbers, and punctuation remain unchanged**.

The project demonstrates practical usage of **Computer Architecture concepts**, including:

- MIPS instruction execution
- Program Counter (PC)
- Instruction Flow & Control Transfer
- 5-Stage MIPS Pipeline
- Memory addressing
- Registers and stack usage
- Procedure calls (jal, jr)

2. Project Objectives

The main objectives are:

- To understand how **high-level encryption logic** is implemented at **assembly level**
- To practice **string manipulation** in MIPS
- To apply **computer architecture concepts** in a real program
- To demonstrate **procedure calls and stack management**
- To observe **instruction execution flow through pipeline stages**



3. Overview of Caesar Cipher

The Caesar Cipher shifts each letter of the message by a fixed number (key).

Example:

- Plain Text: HELLO
- Key: 3
- Cipher Text: KHOOR

4. Program Structure

The program is divided into the following logical parts:

1. Data Segment
2. Main Program Loop
3. Encrypt Mode
4. Decrypt Mode
5. Encryption Subroutine
6. Exit Routine

5. Data Segment (.data)

```
input_buffer: .space 101
```

```
output_buffer: .space 101
```

Concept Covered:

Memory Organization & Addressing

- .space 101 reserves 101 bytes
 - 100 characters
 - 1 null terminator (\0)



- Demonstrates byte-addressable memory
- Used with lb and sb instructions

6. Main Program Loop & Program Counter (PC)

program_loop:

li \$v0, 4

la \$a0, menu

syscall

Computer Architecture Concept: Program Counter (PC)

- The **PC automatically increments** after each instruction
- Sequential execution occurs **unless a branch or jump modifies PC**
- Example control instructions:
 - beq \$t0, 1, encrypt_mode
 - j program_loop

Key Point:

- beq and j **change the PC**
- Demonstrates **control hazards** in pipeline

7. Instruction Flow & Control Transfer

Code Example:

beq \$t0, 1, encrypt_mode

beq \$t0, 2, decrypt_mode

beq \$t0, 3, exit_program

Concept Covered:

Instruction Flow Control

- Branch instructions decide **next PC value**
- Shows **decision-based execution**
- MIPS uses **branch comparison in EX stage**



8. Encrypt Mode (User Input & Key Handling)

```
li $v0, 8  
la $a0, input_buffer  
li $a1, 101  
syscall
```

Concepts Covered:

- System calls
- Memory-mapped I/O
- Register usage (\$a0, \$a1, \$v0)
- Input stored sequentially in memory

9. Decrypt Mode (Negative Key Concept)

```
sub $s0, $zero, $s0  
jal encrypt_message
```

Reuse of Logic

- Decryption is done by **negating the key**
- Same encryption procedure is reused
- Demonstrates **efficient program design**

10. Procedure Call Mechanism

Procedure Call:

```
jal encrypt_message
```

Procedure Return:

```
jr $ra
```

Concepts Covered:

- **Jump and Link (jal)**
- **Return Address Register (\$ra)**
- Procedure-based modular programming



11. Stack & Register Saving

addi \$sp, \$sp, -12

sw \$ra, 0(\$sp)

sw \$s1, 4(\$sp)

sw \$s2, 8(\$sp)

Concept Covered:

Stack Memory & Calling Convention

- Stack grows **downwards**
- Ensures **data integrity**
- Required for nested procedure call

12. Core Encryption Logic (Highlighted Part)

Uppercase Letter Encryption:

sub \$t0, \$t0, 65

add \$t0, \$t0, \$s0

bge \$t0, 26, wrap_upper_pos

blt \$t0, 0, wrap_upper_neg

addi \$t0, \$t0, 65

Lowercase Letter Encryption:

sub \$t0, \$t0, 97

add \$t0, \$t0, \$s0

bge \$t0, 26, wrap_lower_pos

blt \$t0, 0, wrap_lower_neg

addi \$t0, \$t0, 97

Concepts Covered:

- Arithmetic operations
- Conditional branching
- ASCII manipulation



- Modular arithmetic (wrap-around)

13. 5-Stage MIPS Pipeline

The 5 Stages:

Stage	Description	Code Example
IF	Instruction Fetch	PC fetches <code>lb \$t0, 0(\$s1)</code>
ID	Instruction Decode	Register <code>\$s1</code> decoded
EX	Execute	Address calculation
MEM	Memory Access	Byte loaded from memory
WB	Write Back	Value stored in <code>\$t0</code>

Example Instruction:

```
lb $t0, 0($s1)
```

This instruction passes through **all 5 stages**, demonstrating pipeline execution.

14. Instruction Counter & Loop Execution

```
encrypt_loop:
```

```
    lb $t0, 0($s1)
```

```
    beqz $t0, encrypt_done
```

Concept Covered:

Instruction Count Growth

- Loop executes **once per character**
- Longer strings → higher instruction count
- Important for **performance analysis**

15. Memory Access (Load & Store)

```
lb $t0, 0($s1)
```

```
sb $t0, 0($s2)
```

Concepts Covered:

- Load-Store architecture
- Byte-level memory access
- Separate data and instruction memory (Harvard concept)



16. Exit Routine

li \$v0, 10

syscall

Concept Covered:

- Program termination

17. Conclusion

This project successfully demonstrates how **computer architecture concepts** are applied in a **real MIPS assembly program**. The encryption/decryption logic provides a meaningful example of:

- Instruction execution
- Memory manipulation
- Pipeline behavior
- Control flow management

It bridges the gap between **theory and practice**, making it an ideal **Computer Architecture course project**.